GOLD: Interfaces (Fri Oct 2)

Midterm 1 Retakes

Each of the four midterms in CS 18000 will have a retake attempt the following Wednesday (two weeks later for Midterm 4). We will use the 6:30-7:30 and 8:00-9:00 time slots again.

The retake is optional and only your best score counts, so it cannot possibly hurt you.

Note: you are ineligible for the retake if you received a 0 for academic dishonesty.

Sign up for the retake here by Monday, October 5 at 12 pm:
https://edstem.org/us/courses/101720/lessons/184533

Upcoming Test Dates

Wed Oct 7	6:30-7:30 and 8:00-9:00	Midterm 1 Retake
Wed Oct 21	6:30-7:30 and 8:00-9:00	Midterm 2
Wed Oct 28	6:30-7:30 and 8:00-9:00	Midterm 2 Retake
Wed Nov 11	6:30-7:30 and 8:00-9:00	Midterm 3
Wed Nov 18	6:30-7:30 and 8:00-9:00	Midterm 3 Retake
Wed Dec 2	6:30-7:30 and 8:00-9:00	Midterm 4
Wed Dec 16	7:00-8:00 and 8:15-9:15	Midterm 4 Retake

Recap: Monday

Reviewing last time:

- `class Student extends Person`:
  - a Student **inherits** every field and method of Person,
  - and the class holds only the differences.

- One object: the Person parts and the Student parts live in the same location.
  - There is only one arrow!

- Every Student **is a** Person, so `Person p = new Student(...)` compiles.
  - The reverse, `Student s = new Person(...)`, does not.

- A subclass constructor begins by building the parent's part: `super(...)`.

- A subclass can **override** an inherited method (mark it `@Override`) and can still call the parent's version with `super.method()`.

- Whatever the variable's type, **the object's actual type decides** which override runs

Terminology

- `Student` **extends** `Person`:

  - `Student` = **subclass** = child class = derived class

  - `Person` = **superclass** = parent class = base class

- Sub/super feels backwards?
  - The **subclass has more stuff**.
  - Think **sets**, not size:
  - the set of Students is a **subset** of the set of Persons
  - every Student is a Person, not vice versa.

The Classic main

The classic `main`

```
public static void main(String[] args) { … }
```

word	meaning
`public`	any code anywhere may call it: the Java launcher lives outside your class
`static`	belongs to the class, not to an object: when the program starts, **no object exists yet**, so the launcher can only call a static method
`void`	returns nothing
`main`	the name the launcher looks for
`String[] args`	an array of the words typed after the program name on the command line

```
class Main {
    public static void main(String[] args) {
        IO.println("classic main: args.length = " + args.length);
        for (int i = 0; i < args.length; i++) {
            IO.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

Run as `java Main add 5 17.25`, this prints `args.length = 3`, then `add`, `5` and `17.25`.

Everything in `args` is a String: `args[1]` is `"5"`, not `5`.

Pressing Run on a slide supplies no arguments, so `args.length` is 0.

Our `void main()` is the same method without the boilerplate:

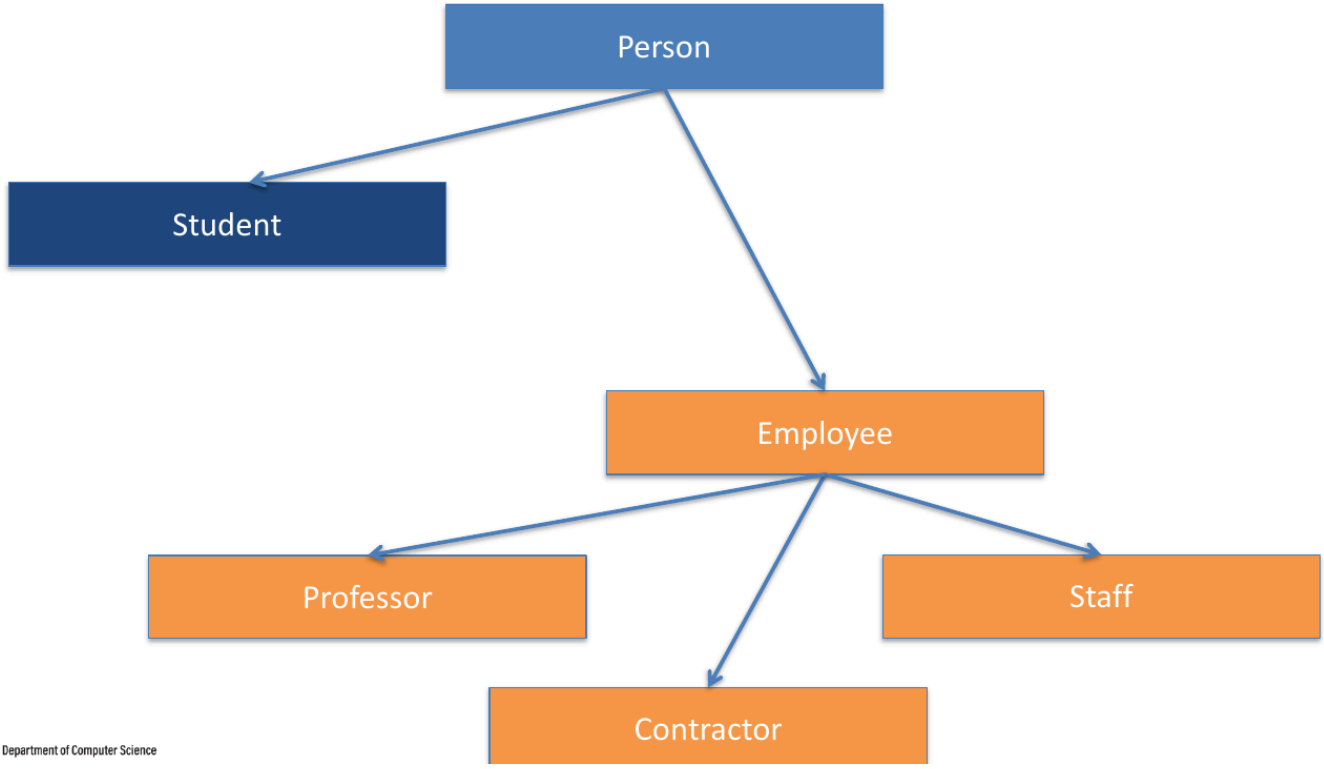
- Java builds an object of your class first and calls `main` on it
- That's why `this.` has been available all semester.

- Both forms are correct.

Object

`Object`: the root of everything

- A class with no `extends` implicitly extends `Object`
- So *every* class descends from `Object`
- one tree, one root
- A class gets exactly **one** `extends` (Java is *single inheritance*)



`Object` gives every class some standard methods.

We'll look at two: `public String toString()` and `public boolean equals(Object o)`

toString

`toString`: always an Override

You may have tried printing an object at some point:

```
class Main {
    void main() {
        Person plain = new Person("Sam", "101 Main St");
        IO.println(plain);            // Person@19d37183, or some such
    }
}

class Person {
    private String name;
    private String address;

    Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    String getName() { return name; }
}
```

What is that output?

`Person@19d37183` is **`Object`'s** `toString()` method, and every class inherits it from the root.

When you wrote your own `toString()` methods, you will be overriding an inherited method.

```
class Main {
    void main() {
        Student s = new Student("Ada", "101 Main St");
        IO.println(s);
    }
}

class Person {
    private String name;
    private String address;

    Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    String getName() { return name; }
}

class Student extends Person {
    Student(String name, String address) { super(name, address); }

    @Override
    public String toString() {
        return "Student(" + getName() + ")";
    }
}
```

```
Student(Ada)
```

Another example: a playing card

Without a `toString()` of its own, printing a `Card` shows something like `Card@7344699f`: the class name and a value called a hash.

- This is Java's default description of an object, and it is useless usually.
- The fix: give the class a `toString()` method.
- `println` and String concatenation call `toString()` automatically:

```
class Main {
    void main() {
        Card c = new Card("Ace", "Spades");
        IO.println(c);                    // Ace of Spades
        IO.println("drew: " + c);         // drew: Ace of Spades
    }
}

class Card {
    private String rank;
    private String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() {
        return rank + " of " + suit;
    }
}
```

`toString` is the object's self-description. Every class you write should have one.

equals: Same Card?

`Object` defines `equals`

With Strings we used `s.equals(t)` to ask whether two Strings hold the same text.

Every class inherits an `equals` method from `Object`, just as it inherits `toString`.

So we can already ask the same question about two Cards.

Pause and predict (poll time)

Q. `Card` is the class from the previous slide: a rank, a suit, a constructor and a `toString`.

```
class Card {
    private String rank;
    private String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() { return rank + " of " + suit; }
}
```

It has **no** `equals` method of its own. What happens here?

```
Card a = new Card("Ace", "Spades");
Card b = new Card("Ace", "Spades");
IO.println(a.equals(b));
```

- **A.** Prints `true`: the two cards have the same rank and suit
- **B.** Prints `false`
- **C.** Compile error: `Card` has no `equals` method
- **D.** It compiles, then crashes at run time

Overriding equals

Overriding `equals`

The `equals` that every class inherits from `Object` is plain `==`:

- it asks only whether the two arrows point at the **same object**.
- To make it mean **same contents**, a class can override it:

```
class Main {
    void main() {
        Card a = new Card("Ace", "Spades");
        Card b = new Card("Ace", "Spades");
        Card k = new Card("King", "Hearts");
        IO.println(a == b);              // false: two objects
        IO.println(a.equals(b));         // true:  same rank and suit
        IO.println(a.equals(k));         // false: different card
        IO.println(a.equals("Ace"));     // false: a String is not a Card
    }
}

class Card {
    private String rank;
    private String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    @Override
    public boolean equals(Object o) {
        if (o instanceof Card c) {
            return this.rank.equals(c.rank) && this.suit.equals(c.suit);
        }
        return false;
    }
}
```

- `equals` returns a `boolean`, and its parameter is an `Object`, not a `Card`:
  - an override must keep the inherited signature,
  - and `@Override` checks that you did.

- So any two objects may be compared regardless of type, and the method's first job is to find out what it was handed.

- `o instanceof Card c` does two things at once:
  - it asks *is* `o` *a Card?* and,
  - when the answer is yes, it hands you the same object as `c`, already typed as a `Card`.

- `instanceof` is the is-a question as an operator.
  - It is `false` for `null`, so `a.equals(null)` is `false`, not a crash.

- Then we compare the fields, each in its own way: Strings and other objects with `.equals`, numbers with `==`.

This test-and-name form is the first example of **pattern matching** in modern Java.



Good to know: `Objects.equals(x, y)` compares two references that might be `null` without crashing.

Limitations of Inheritance

Java's inheritance rule: **one** `extends` per class.

How much of a limitation is this?

Now that we've built some classes, suppose the registrar asks for one loop that returns a budget line item for...

- `Student` : is a `Person`
- `Professor` : is a `Person`
- `Course` : is *not* a Person. Not even close.
- `Building` : also not a Person.

There is no common parent to write the loop against, and we can't invent one

- `Course extends Person` is absurd
- Common superclass `PurdueEntity` or `Payable` ?
  - each class's one `extends` may already be spent, or we may not control all the top-level superclasses.

Inheritance shares **code** and **identity**. These four classes share neither.

They share only an **ability**: each can produce a budget line (wages, maintenance, etc)

Idea: Contracts

Today's idea: a contract

When we wrote our first classes, I gave the following advice for completing assignments:

"read the spec like a **contract**: exact class name, exact method names, exact parameter lists, and the return types."

Today we write that contract down in Java itself:

- an **interface** is a new kind of type in Java
- an interface is a **contract** between classes
- The interface declares the promise: "these methods will exist."
- Any class may sign the contract by providing those methods.
- We write code that works with **anything that signed** the contract (and nothing else about the signer matters).
- That's polymorphism!

Interface

Interface syntax: all promises, no code

```
public interface Payable {
    double weeklyPay();      // headers only —
    String payTo();          // no bodies, no fields
}
```

- Declared like a class and lives in its own file
- Note the word `interface` instead of `class`
- But method headers only (don't forget the `;`)
- No method bodies and no fields. (The extra reading "The fine print" at the end has the exceptions.)
- Nothing to construct: `new Payable()` is meaningless
- An interface is a class that's all promise and no code

Implements

`implements`: signing the contract

```
class Main {
    void main() {
        Coach c = new Coach("Knuth");
        IO.println(c.payTo() + ": " + c.weeklyPay());
    }
}

interface Payable {
    double weeklyPay();
    String payTo();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    public Coach(String name) { super(name); }

    @Override
    public double weeklyPay() { return 2400.0; }

    @Override
    public String payTo() { return getName(); }
}
```

- `implements Payable`: this class **signs** the contract
- it must now provide **every** promised method with `public` access
- `@Override` works here too (you're overriding the promise)
- Note the header: `extends` AND `implements` **together**

What happens if you forget a method?

```
class Main {
    void main() {
        IO.println(new Broken().payTo());
    }
}

interface Payable {
    double weeklyPay();
    String payTo();
}

class Broken implements Payable {
    public String payTo() { return "Ada"; }
    // forgot weeklyPay()
}
```

Pause and predict (poll time)

Q. `Broken` signs the `Payable` contract and forgets `weeklyPay()`.

What happens when we press Run on the snippet above?

- **A.** It runs; `weeklyPay()` on a `Broken` would return `0.0`
- **B.** Compile error: `Broken` does not provide `weeklyPay()`, which `Payable` promised
- **C.** It runs, and crashes only if `weeklyPay()` is called
- **D.** It runs; `Broken` is simply not treated as a `Payable`

Interface Types

A promise is a type

Last time: `Person p = new Student(...)`.

The same idea works with a **contract** (interface) as the type:

```
Payable p = new Coach("Knuth");     // variable typed by the PROMISE
IO.println(p.weeklyPay());
```

And we can write polymorphic code for the budget system

```
Payable[] payroll = { new Coach("Knuth"), new Building("Matthews") };
for (Payable p : payroll)
    IO.println(String.format("%s: $%.2f", p.payTo(), p.weeklyPay()));
```

```
Knuth: $2400.00
Physical Facilities: $800.00
```

A `Coach` is a `Person`, but a `Building` is related to nothing.

```
class Main {
    void main() {
        Payable[] payroll = { new Coach("Knuth"), new Building("Matthews") };
        for (Payable p : payroll) {
            IO.println(String.format("%s: $%.2f", p.payTo(), p.weeklyPay()));
        }
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

Pause and predict (poll time)

Q. `Coach extends Person implements Payable`, as above: `getName()` comes from `Person`, and `Payable` promises `weeklyPay()` and `payTo()`. Which line does **not** compile?

```
Payable p = new Coach("Knuth");     // line 1
IO.println(p.weeklyPay());          // line 2
IO.println(p.getName());            // line 3
Person q = new Coach("Hopper");     // line 4
```

- **A.** Line 1
- **B.** Line 2

- **C.** Line 3
- **D.** Line 4

Casting

Casting objects

We saw `p.getName()` did not compile. Why?

The object is a `Coach` and it has a name, but through a `Payable` variable we may call only what `Payable` promises.

Remember casting numbers? `(int) 8.99` is `8` .

The same syntax works on objects:

```
class Main {
    void main() {
        Payable p = new Coach("Knuth");
        Coach c = (Coach) p;             // a cast: "trust me, this Payable is a Coach
        IO.println(c.getName());
        IO.println(c == p);              // one object, two variables

        Payable q = new Building("Matthews");
        Coach d = (Coach) q;             // this compiles too. Is it true?
        IO.println(d.getName());
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

What we get:

```
Knuth
true
Exception in thread "main" java.lang.ClassCastException:
    class Building cannot be cast to class Coach
```

- A cast on an object **converts nothing**.

    - `(int) 8.99` made a new value;
    - `(Coach) p` **does not make a new object**.
    - `c` and `p` are two arrows to the same Coach.
    - The cast only gives us a variable of a different type, and the variable's type decides what we are allowed to ask.

- The compiler takes our word for it. Java **checks the promise when the program runs**, against the real object.
    - The first cast is true.
    - The second is not: a `ClassCastException`, and the program stops.

- Every `Coach` is a `Payable`, so `Payable p = new Coach(...)` needs no cast and cannot fail.
- The reverse needs a cast, and it can fail: not every `Payable` is a `Coach`.

Avoid crashes: ask first

We already know how to ask: `instanceof`, in the test-and-name form from `equals`.

```
class Main {
    void main() {
        Payable[] payroll = { new Coach("Knuth"), new Building("Matthews") };
        for (Payable p : payroll) {
            if (p instanceof Coach c) {              // is p a Coach? then call it c
                IO.println("coach: " + c.getName());
            } else {
                IO.println("not a coach: " + p.payTo());
            }
        }
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

```
coach: Knuth
not a coach: Physical Facilities
```

`p instanceof Coach c` is the question and the cast in one step, so it cannot fail. Prefer it. A bare cast is for when we are certain what the object is.

Example: Game Design

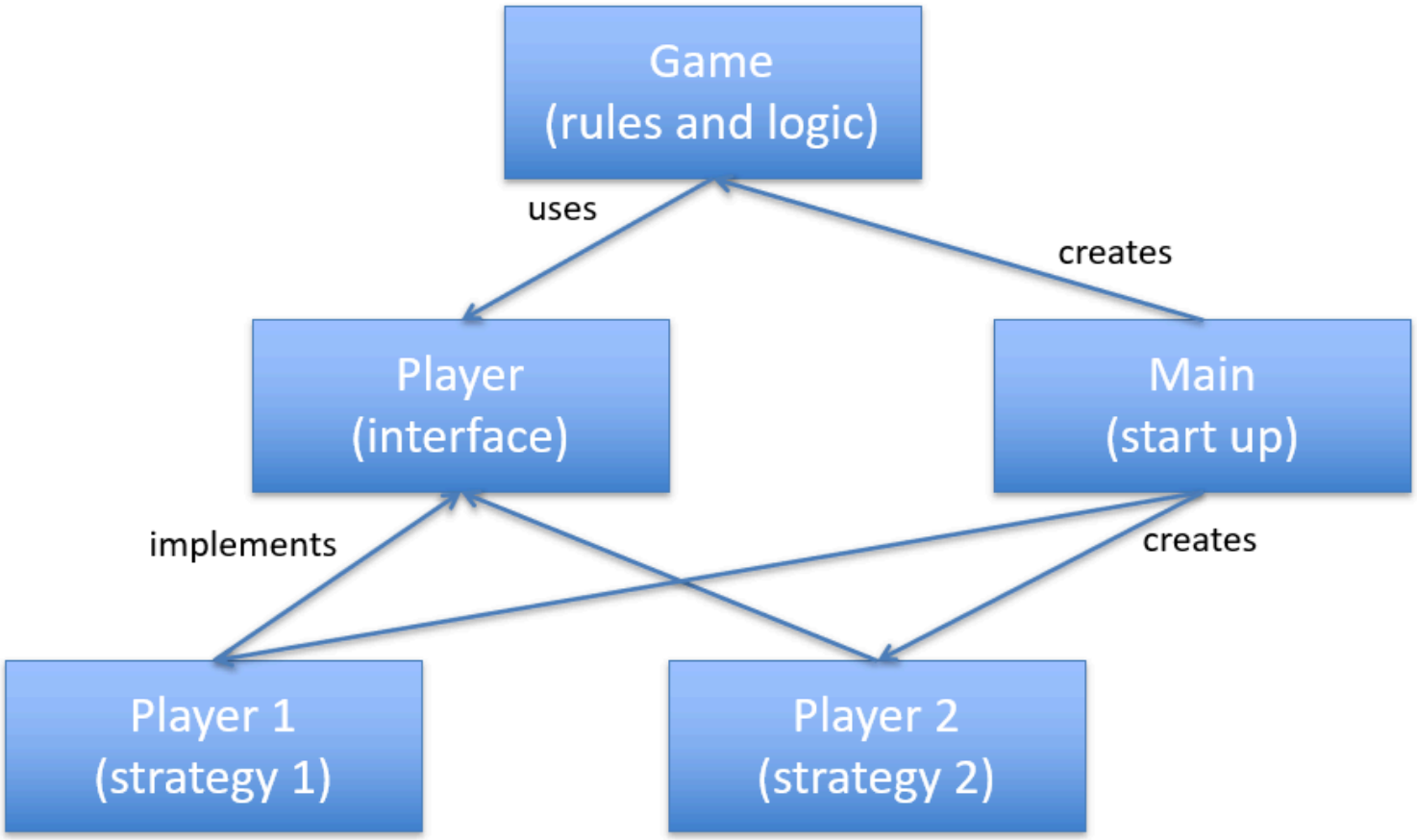
Example: Building a Game

Problem: Implement a turn-based game in which fantasy-inspired creatures battle over items in a dungeon.

Multiple computer-controlled players, each with own strategy

Rules enforced by game controller, a class which orchestrates the game and its battles (basically Nim)

The design structure: `Game` will be written against the `Player` contract without needing to know which player classes exist.



Use of Java interface:

- Each player class (`Dragon` etc) implements Player interface
- Game controller `Game` expects parameters of type Player

Main program:

- Creates player objects from classes

- Creates game controller with player objects (so we can play more than once without restarting program)
- Starts game controller
- Prints results

Game Players

The rules, and the contract

The game: a pile of items.

- Players alternate turns, each taking 1, 2, or 3 items.
- Whoever takes the last item wins.

Everything `Game` needs from a player abstracted into a contract:

```
interface Player {
    String getName();
    int takeSome(int itemsLeft);   // your move: how many to take (1..3)
}
```

A player needs to model this abstraction: a name and a strategy.

Two players, two personalities

```
class Main {
    void main() {
        Player d = new Dragon();
        Player b = new Butterfly();
        IO.println(d.getName() + " would take " + d.takeSome(10));
        IO.println(b.getName() + " would take " + b.takeSome(10));
    }
}

interface Player {
    String getName();
    int takeSome(int itemsLeft);   // your move: how many to take (1..3)
}

class Dragon implements Player {
    public String getName() { return "Dragon"; }
    public int takeSome(int itemsLeft) { return 3; }     // greedy: max every time
}

class Butterfly implements Player {
    public String getName() { return "Butterfly"; }
    public int takeSome(int itemsLeft) { return 1; }     // cautious: minimum
}
```

Completed Game

`Game` : written against the contract

Here's the `Game` class.

We call it the *controller* because it runs each game. The game state is the variable `int items` inside `play`

```
class Game {
    private Player p1, p2; // the INTERFACE type
    Game(Player p1, Player p2) { this.p1 = p1; this.p2 = p2; }

    void play(int items) {
        Player cur = p1;
        while (items > 0) {
            int take = Math.min(Math.max(cur.takeSome(items), 1), Math.min(3, items));
            items -= take;
            IO.println(cur.getName() + " takes " + take + ", " + items + " left");
            if (items == 0) IO.println(cur.getName() + " wins!");
            cur = (cur == p1) ? p2 : p1;
        }
    }
}
```

`Game` knows only the promise, not `Dragon` or `Butterfly`

Full game

```
class Main {
    void main() {
        new Game(new Dragon(), new Butterfly()).play(10);
    }
}

class Game {
    private Player p1, p2; // the INTERFACE type
    Game(Player p1, Player p2) { this.p1 = p1; this.p2 = p2; }

    void play(int items) {
        Player cur = p1;
        while (items > 0) {
            int take = Math.min(Math.max(cur.takeSome(items), 1), Math.min(3, items));
            items -= take;
            IO.println(cur.getName() + " takes " + take + ", " + items + " left");
            if (items == 0) IO.println(cur.getName() + " wins!");
            cur = (cur == p1) ? p2 : p1;
        }
    }
}

interface Player {
    String getName();
    int takeSome(int itemsLeft);   // your move: how many to take (1..3)
}

class Dragon implements Player {
    public String getName() { return "Dragon"; }
    public int takeSome(int itemsLeft) { return 3; }     // greedy: max every time
}

class Butterfly implements Player {
    public String getName() { return "Butterfly"; }
    public int takeSome(int itemsLeft) { return 1; }     // cautious: minimum
}
```

Extended Game

PerfectPete has entered the chat

A new class written after `Game` was finished and closed:

```
class PerfectPete implements Player {
    public String getName() { return "PerfectPete"; }
    public int takeSome(int itemsLeft) {
        int m = itemsLeft % 4;          // leave the opponent a multiple of 4
        return (m == 0) ? 1 : m;
    }
}
```

```
new Game(new PerfectPete(), new Dragon()).play(10);
```

Pete plays the `% 4` strategy. Who do you think is going to win?

Full code:

```
class Main {
    void main() {
        new Game(new PerfectPete(), new Dragon()).play(10);
    }
}

class Game {
    private Player p1, p2; // the INTERFACE type
    Game(Player p1, Player p2) { this.p1 = p1; this.p2 = p2; }

    void play(int items) {
        Player cur = p1;
        while (items > 0) {
            int take = Math.min(Math.max(cur.takeSome(items), 1), Math.min(3, items));
            items -= take;
            IO.println(cur.getName() + " takes " + take + ", " + items + " left");
            if (items == 0) IO.println(cur.getName() + " wins!");
            cur = (cur == p1) ? p2 : p1;
        }
    }
}

interface Player {
    String getName();
    int takeSome(int itemsLeft);   // your move: how many to take (1..3)
}

class Dragon implements Player {
    public String getName() { return "Dragon"; }
    public int takeSome(int itemsLeft) { return 3; }     // greedy: max every time
}

class Butterfly implements Player {
    public String getName() { return "Butterfly"; }
    public int takeSome(int itemsLeft) { return 1; }     // cautious: minimum
}

class PerfectPete implements Player {
    public String getName() { return "PerfectPete"; }
    public int takeSome(int itemsLeft) {
        int m = itemsLeft % 4;          // leave the opponent a multiple of 4
        return (m == 0) ? 1 : m;
    }
}
```

We never edited `Game`.

That is what a contract (`interface`) gives us:

- code written against the interface is finished
- growth (development) happens by adding new signers.

Subclasses vs Interfaces

Which tool, when?

You want to share:	Use	The question it answers
code + identity (fields, method bodies, is-a-kind-of)	`extends` a superclass	"what **is** this thing?"
ability only (functionality across unrelated classes)	`implements` an interface	"what can this thing **do**?"

- `Student` **is a** `Person` → inheritance
- `Coach`, `Building` can both be paid → interface
- And it's not either/or: `class Professor extends Person implements Payable, Comparable<Professor>` is everyday Java

Closing

One last question

The big idea: code written against an interface is finished, and a program grows by adding new signers.

Pause and predict (poll time)

Q. In the game: `interface Player` promises `getName()` and `takeSome(int)`.

`Dragon`, `Butterfly` and `PerfectPete` implemented it, and `Game` is written against `Player`.

Which **one** of these changes would stop classes that already exist from compiling?

- **A.** Adding a new class `Mimic implements Player` that provides both methods
- **B.** Changing `Dragon`'s `takeSome` to return `2`
- **C.** Adding a new method `boast()` to `Butterfly` only
- **D.** Adding a new method `int lastMove();` to the `Player` interface

Extra Reading: final

`final`: Fields, Methods, Classes

The author of a class can say "*this may not change*" in three places.

`final` fields

A card never changes what it is.

Mark the fields `final` and Java enforces it:

- `final` fields must be assigned **exactly once, in the constructor**,
- and can never change after:

```
class Main {
    void main() {
        Card c = new Card("Ace", "Spades");
        IO.println(c.getRank() + " of " + c.getSuit());
    }
}

class Card {
    private final String rank;   // "Ace", "2", ..., "King"
    private final String suit;   // "Hearts", "Spades", ...

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String getRank() {
        return rank;
    }

    public String getSuit() {
        return suit;
    }
}
```

A card cannot change, so we have getters, but no setters.

An object like this is **immutable**, like `String`.

Compare the Die: `roll()` *mutates* its state. Both designs are correct for their objects.

Note: a setter on a final field is a compile error:

```
public void setRank(String r) {
    this.rank = r;      // COMPILE ERROR: cannot assign a value to final variable rank
}
```

`final` methods

A method marked `final` **cannot be overridden**.

The author of the parent class decides which behavior a subclass may replace.

On Monday the loaded die cheated by overriding `roll()`. The author of `Die` can forbid that:

```
class Die {
    private Random rng;
    private int numFaces;
    private int state;

    // ... constructor as before ...

    public final int roll() {              // final: no subclass may override this
        state = 1 + rng.nextInt(numFaces);
        return state;
    }
}

class LoadedDie extends Die {
    @Override
    public int roll() {                    // COMPILE ERROR
        return 6;
    }
}
```

```
error: roll() in LoadedDie cannot override roll() in Die
  overridden method is final
```

`final` classes

A class marked `final` **cannot be extended** at all:

```
final class Card {                          // final: no class may extend this
    // ... as above ...
}

class Joker extends Card {                  // COMPILE ERROR: cannot inherit from final
}
```

`String` is a final class: `class LoudString extends String` does not compile (`cannot inherit from final String`).

`final` on a…	means
field	assigned exactly once, in the constructor, and never again
method	no subclass may override it
class	no class may extend it

Extra Reading: Multiple Interfaces

Extra reading: not presented in lecture.

Multiple Interfaces

A class can implement multiple interfaces. As many as we want!

The methods that must be implemented are the union of the methods specified in the interfaces

Example:

```
class Main {
    void main() {
        Horse h = new Horse();
        h.mount();
        h.train();
        Rideable r = h;      // a Horse is Rideable
        Trainable t = h;     // ...and Trainable
        IO.println("one horse, two contracts");
    }
}

interface Rideable { void mount(); }
interface Trainable { void train(); }

class Horse implements Rideable, Trainable {
    public void mount() { IO.println("saddled up"); }
    public void train() { IO.println("learned a trick"); }
}
```

Think about this: What if we say `r.train()` in `main`?

Extra Reading: The Fine Print

Extra reading: not presented in lecture.

Reference: the fine print

- **Interfaces can carry code**
  - `default` methods (a body all signers inherit)
  - `default String getFullAddress() { return getStreet() + ", " + getCity(); }`
  - and `static` helpers.
  - They exist so published libraries can grow without breaking their signers
  - Recognize; but don't reach for it as a first option.
- **Fields in interfaces**
  - are implicitly `public static final` ; i.e. constants.
  - The old trick of `MyClass implements SomeConstants` just to import names is a **named antipattern**
  - (it welds an implementation convenience into your public type, forever).
  - Constants should live in classes; reference them as `Constants.X`.
- `Iterable` is the contract behind every for-each loop you've written.
  - We won't cover it in CS 18000 (old-fashioned and confusing).
  - Now you know why sometimes you can write `for (SomeType value : collection)` and sometimes you can't.
  - Hand-implementing it correctly is beyond the scope of our class.

Take Home: A New Player

Today's game: `Player`, `Game`, `Dragon` and `Butterfly` are given, complete. Do not change them: the point of a contract is that `Game` is finished.

Part 1: a new player. Finish **`Halver`**, a player who takes half of what is left.

- As given to you, the program does not compile. Press **Run** and read the error: `Game` wants a `Player`, and `Halver` has not signed the contract.
- Sign it (`implements Player`) and press **Run** again. The next error is the one from today's poll. Read it too.
- Keep the two promises. `getName()` returns `Halver`. `takeSome(itemsLeft)` returns half of `itemsLeft`, rounded down, but never fewer than 1 and never more than 3. So a pile of 5 gives 2, a pile of 10 gives 3, and a pile of 1 gives 1.

Part 2: code written against the contract. In `Referee.java`, write **`boldest(Player[] players, int itemsLeft)`**: the name of the player who would take the most from a pile of that size. On a tie, the first of them in the array wins.

Your method may use only what `Player` promises; one test hands it a player class it has never seen.

Run plays one game. Press **Test** to run the tests.